



Faculty of Science

Liam Green

Data Science Toolbox (MATHM0029)

PORTFOLIO

December 2025

School of Mathematics

Contents

1 Portfolio questions	1
1.1 Question R02.3	1
1.2 Question R04.1	2
1.3 Question R07.1	3
1.4 Question R10.3	4
References	6
A Lemmas	8
A.1 Random k -fold cross-validation bias in forecasting	8
A.2 Asymptotic behaviour of expected mean squared error	10
B Figures	11
B.1 Time series diagnostic plots	11
B.2 Blocked cross-validation model comparison	12
B.3 Model forecast comparison	13
B.4 k -nearest neighbour regression with optimal k	14
B.5 Universal Approximation Theorem results	15
C R code	19
C.1 Blocked cross-validation	19
C.2 k -nearest neighbour regression with optimal k	20

Chapter 1

Portfolio questions

Question R02.3

We represent the data as a time series $\{y_t\}_{t=1}^T$ to, now, account for serial dependence on time, where y_t are the recorded monthly temperature anomalies. In this forecasting problem, we aim to naturally predict future monthly temperature anomalies based solely on past observations. A poignant assumption of random k -fold cross-validation (CV) is that errors must be independent and identically distributed [1]. This assumption, however, is largely violated in time series modelling. For observations following a stochastic process $\{y_t\}_{t=1}^T$, adjacent observations are correlated, since $\text{Cov}(y_t, y_{t+\tau})$, for some lag $\tau > 0$. The CV procedure proposed in the workshop is thus infeasible as it does not respect the temporal order of our data, leading to inflated prediction errors (see Lemma A.1 for a proof). To circumvent this issue, we explore the use of *blocked* CV [2] to compute out-of-sample errors without leaking future information into the past. This involves partitioning the series into a number of contiguous, disjoint ‘training’ blocks (or *windows*) of fixed length. Within each block, a forecast is made, h steps ahead of the training region. Predictions are then compared to the actual h observations superseded by the test region.

To evaluate predictive accuracy and uncertainty, we apply blocked CV across a range of forecast horizons $h \in \{1, 12, 60, 120, 295\}$, for both a polynomial regression model of degree 5 (used in the workshop) and an ARIMA(2, 1, 1)(0, 1, 2)₁₂ [3] model – see Figure B.1 for brief justification of this model choice. With $n = 1997$ observations, we divide the series into 5 windows to ensure ample training data in each. We fix $h = 295$ to represent the number of months from the last observation (in May 2016) to December 2040. We parallelise the computation of 3 performance metrics for each horizon and benefit from an accelerated ARIMA model fitting process (on a high-dimensional data set) as a result. Appendix C.1 displays the code for use.

As shown in Figure B.2, the ARIMA model consistently outperforms across all metrics: mean absolute error (MAE), root mean squared error (RMSE) and the empirical standard deviation of forecast errors (SD_Uncertainty). At long horizons, e.g. $h = 295$, the polynomial model suffers from large extrapolation errors (MAE ≈ 15.96 , RMSE ≈ 14.49), whilst ARIMA remains stable (MAE ≈ 0.22 , RMSE ≈ 0.27). Attributable to its differencing and seasonal components, this highlights ARIMA’s robustness for long-range forecasting; the polynomial model exhibits a steep rise in standard error uncertainty over time, indicating much poorer forecasting reliability. Furthermore, Figure B.3 compares the long-range forecasts of the polynomial regression and ARIMA models up to the end of 2040. While both models predict continued increases in monthly temperature anomalies, the polynomial model projects a far more exponential trajectory, reaching anomalies exceeding 2.5°C by 2040. In contrast, ARIMA produces a more moderate and linear increase, with fanning prediction intervals over time. Compared to the polynomial model’s oversensitivity to trends in the data, especially at the boundaries, this underlines the ARIMA model’s more conservative forecasting behaviour.

Our ‘blocked’ approach is often favoured for its balanced bias–variance trade-off. It enables efficient use of the full dataset while remaining less computationally demanding than alternative schemes (e.g. rolling window) [4]. This is apparent with data containing a large number of observations, such as ours. However, the use of non-overlapping blocks reduces the number of training samples in each window, potentially reducing the validity of our error estimates. This particular implementation also does not fully account

for autocorrelation across the block boundaries (i.e. between training and test blocks), which could still bias error estimates if not mitigated by a buffer region. Nonetheless, the risk is naturally lower than other methods that allow windows to overlap (e.g. expanding window). Since blocked CV imposes a fixed temporal decomposition, uncertainty estimates can vary notably with the number and alignment of blocks. For instance, one can show that increasing the number of windows from 5 to 10 causes explosive growth in estimates for the polynomial model's 2040 forecast. Moreover, blocks may not align with key trends: historic 19th-century data may poorly represent contemporary warming, distorting CV estimates due to increased variance. Overall, a well-rounded sensitivity analysis on our hyperparameter choices that also incorporates other time series CV strategies will enable a more holistic evaluation of forecast reliability and interval accuracy.

Question R04.1

In the k -nearest neighbour (kNN) approach, the choice of hyperparameter k is inevitably subjective in practice: a small k leads to highly local yet noisy predictions with large variance, whilst a large k averages over distant points and inflates bias. A common strategy is to perform CV over a range of candidate values and select $k = k^*$ minimising the mean squared error (MSE). An odd value of k is often chosen to avoid ties in distance calculations. Although straightforward to implement, the search space for k is heuristic and may not contain an 'optimal' result.

Classical nonparametric theories [6] that scrutinise the choice of k impose stringent conditions on the risk function (e.g. Lipschitz continuity and McDiarmid's concentration inequalities). These assumptions, while mathematically sound, remain highly theoretical and often inappropriate for practical applications. For instance, Zhao and Lai devise an approach in [7], whereby k is selected adaptively to successfully attain the minimax lower bound; unfortunately, this requires optimisation of three hyperparameters dependent on the selection of training samples.

Another poignant downfall of kNN is that it suffers majorly from the 'curse of dimensionality'. By Lemma A.2, the expected MSE grows quadratically with dimension $d \geq 3$. For a neighbourhood with radius $r_k(\mathbf{x}) \propto (k/n)^{1/d}$, locality requires $r_k(\mathbf{x}) \rightarrow 0$. This condition is only satisfied if $\exp(\log(k/n)/d) \rightarrow 0 \Leftrightarrow \log(n/k)/d \rightarrow \infty$. For $\log(n/k) > d$, this is well-defined, thus n must scale exponentially with d to ensure universal consistency. In practice, this is difficult to achieve for high-dimensional data sets, so the choice of k is, again, likely to be suboptimal. Kpotufe challenges this in [8] by proving that the minimax bound can be attained based on the intrinsic dimension of the predictor, however its true value is unknown and the choice of k also necessitates further hyperparameter tuning. Literature highly concentrated on asymptotic error bounds has, as a result, offered little insight into applicable estimations for kNN, particularly with finite sample sizes.

A key advancement was recently made by Azadkia, who established the first finite-sample, non-asymptotic framework for selecting a near-optimal value of k in kNN regression, involving leave-one-out cross-validation (CV) under much weaker assumptions [9]. Her result provides an explicit upper bound on the predicted MSE that holds for any sample size n and any dimension d , converging efficiently without extra design parameters or distributional constraints. Interestingly, the regression function need not be smooth (Lipschitz continuity is no longer assumed) as the bound does not depend on it. Ultimately, this makes Azadkia's approach particularly attractive in practice.

Note that, throughout, it is assumed that k is allowed to grow in a way such that, as $k \rightarrow \infty$, $k/n \rightarrow 0$ as $n \rightarrow \infty$. What happens if k is fixed? We follow Azadkia's work and assume that, for $y = m(\mathbf{x}) + \epsilon$, $\mathbb{E}[\epsilon] = 0$ and $\text{Var}(\epsilon) = \sigma^2 < \infty$:

$$\begin{aligned} \text{Var}(\hat{m}_{k,n}(\mathbf{x}_i)) &= \text{Var}\left(\frac{1}{k} \cdot \sum_{j \in \mathcal{N}_k(i)} (m(\mathbf{x}_j) + \epsilon_j)\right) \\ &= \text{Var}\left(\frac{1}{k} \cdot \sum_{j \in \mathcal{N}_k(i)} \epsilon_j\right) \\ &= (1/k^2) \cdot k\sigma^2 \\ &= \sigma^2/k, \end{aligned}$$

where the second equality emanates from the fact that $\mathbf{x}$ is deterministic and, thus, has zero conditional variance. Unless $k \rightarrow \infty$, the variance persists and the estimator never converges to Bayes' risk.

We utilise the `knnopt` function [10], kindly formatted by Azadkia herself, on the data from Lecture 4.1 to predict the response-byte behaviour as a (nonparametric) function of the originating byte size:

$$m(\mathbf{x}) = \mathbb{E}[\text{resp_bytes} \mid \text{orig_bytes} = \mathbf{x}].$$

For computational purposes, we restrict the data to successful flows, remove outliers based on Mahalanobis distance and consider $k \in \{1, \dots, 100\}$. Our first pass of the algorithm detects an optimal value of $k^* = 8$, corresponding to a MSE value of 0.1432735. We further assess the stability of this method via bootstrapping using 100 samples; the resulting distribution of $k = k^*$ is concentrated around a mean of approximately 6, with a median of 6 and an interquartile range [6, 7]. This indicates that the neighbourhood size selected by LOOCV is reasonably stable across resamples despite the high level of noise and heteroscedasticity present in the data. Figure B.4 displays the fitted regression curve and Appendix C.2 shows the R code used for analysis.

Question R07.1

The Universal Approximation Theorem (UAT) establishes that feed-forward neural networks with (at least) one hidden layer (a multilayer perceptron, or MLP) can have remarkable representational power. In its classical form, Cybenko proved that a single-hidden-layer network with a sigmoidal activation can approximate any continuous function on a compact domain to arbitrary accuracy [11]. This result was later generalised by Hornik in [12] to all “continuous, bounded and nonconstant” activation functions. Ultimately, these results formalise the idea that even shallow networks (those with only one hidden layer) are universal approximators [13]. However, universality concerns only the existence of appropriate parameters – it says nothing about whether gradient descent can find such parameters, how many neurons are required or whether shallow architectures are computationally efficient.

Modern research, such as that of Poggio et al. in [14], expands by proving that, while shallow networks are universal, they may require many parameters to approximate certain high-dimensional functions. Deep networks, by contrast, can exploit hierarchical compositional structure to achieve similar approximation accuracy, thereby avoiding the ‘curse of dimensionality’. So, although a shallow network with sufficient width can, in principle, represent a wide range of functions, deep networks often provide practical advantages in learnability and parameter efficiency.

For this task, we compare six MLP models of varying width and depth on the FashionMNIST dataset: shallow networks (one hidden layer) and deeper networks (2–3 layers), with widths varying from 16 to 256 units. All models are trained from scratch with identical preprocessing (flattening, normalisation) and settings (cross-entropy loss, Adam optimisation with $\text{lr} = 1 \times 10^{-3}$, batch size 32) to maintain consistency. Training lasts for 15 epochs and the training/test loss and accuracy are reported for each network. The best model is then assessed via confusion matrix and prediction examples. The corresponding `python` code file is contained separately (`r07.1.ipynb`). See Figures B.5–B.8 for a visualisation of the results discussed below.

Shallow networks – especially the 1×128 and 1×256 models – achieve the highest final test accuracies ($\sim 89\%$) and lowest final training losses (~ 0.1). In contrast, a deeper $16 \times 16 \times 16$ network finishes with a higher training loss (~ 0.28). This shows that substantial depth is not essential and a wide shallow network can learn an effective decision boundary on a moderately complex dataset such as FashionMNIST, consistent with the UAT.

However, despite their superior accuracy, the shallow models consistently show higher test loss (e.g., 1×128 reaches ~ 0.51), whereas deeper networks, such as $16 \times 16 \times 16$, achieve lower test loss (~ 0.39). This reflects how the theorem guarantees representational capacity, but not training efficiency or probability calibration. Shallow models often produce less calibrated outputs and cross-entropy strongly penalises low-confidence predictions, even when classifications are correct. Parameter counts further highlight this. The $16 \times 16 \times 16$ model utilises only 13,274 parameters, whereas the 1×256 model uses 203,530 – yet their accuracies differ by only a modest 9.6 percentage points. This suggests that deeper models can, theoretically, match performance with far fewer parameters and yield smoother, better-calibrated decision boundaries, as per the refinements of the UAT proposed by Poggio et al.

Although the 1×256 network attains slightly higher accuracy, the 1×128 model performs nearly identically with vastly less parameters. Favouring simplicity under Occam’s razor [15], we select the 1×128 MLP as the final model. Its confusion matrix shows most errors occur between visually similar classes (e.g., shirt vs. t-shirt/top). Qualitative prediction samples confirm this, showcasing that the model confidently identifies distinct items such as sandals and trousers, with occasional mistakes in classes with similar silhouettes.

Question R10.3

General matrix-matrix multiplication (GEMM) is a central example of how algorithmic ‘work’ interacts with hardware constraints. Fatahalian et al., in particular, explore (square) dense matrix-matrix multiplication (DGEMM) in [16]. Its naive $O(n^3)$ cost follows directly from the triple-nested loop, where each of the n^2 outputs performs an inner product of length n , consistent with the notion of work in [17]. On central processing units (CPUs), Lectures 10.1–2 tell us that vectorisation exploits *temporal locality* (the reuse of recently accessed data) and single-instruction multiple-data (SIMD) units to accelerate many independent map operations. When locality is poor, however, DGEMM becomes bandwidth-bound, as operands must be fetched repeatedly from slower memory. Graphics processing units (GPUs), by contrast, expose massive parallelism due to a vastly higher number of cores (thus higher *spatial locality*). Despite this, the paper demonstrates that GPU-based DGEMM is exacerbated by the rate at which data can be delivered to such units due to technological constraints at the time. The evolution of the single-instruction multi-thread (SMT) execution model has enabled the development of contemporary architectures providing far more impressive benchmarks [18].

Efficient CPU DGEMM calculations involve restructuring computations so that each loaded tile is reused heavily in fast L1/L2 cache. Fatahalian et al. show in Figure 2 that ATLAS achieves $\sim 65\%$ of peak floating point operations (FLOPs) on a Pentium 4 for 1024×1024 DGEMM – a superior result the authors attribute to ‘blocking’, which keeps submatrices resident in cache long enough to amortise memory-access costs. Of course, this process directly benefits structured procedures (singular value decomposition, principal component analysis, QR factorisation) important in areas such as dimensionality, since each relies on repeated multiplications of dense matrix tiles that can be efficiently reused in cache. Within the map-reduce framework proposed in the lectures, each row of matrix A and each column of matrix B participate in many independent multiply-accumulate steps before reduction, whereby blocking ensures that this occurs while tiles remain in fast memory. Ortega illustrates this locality principle in [19], demonstrating up to $16\times$ speedup for larger matrices once block reuse dominates communication. It is worth noting, however, that this locality advantage is amplified by the architectural transparency and tuning capabilities of a CPU. ATLAS, for example, probes cache sizes and selects tile shapes accordingly (p.2 of [16]). As stated by Fatahalian et al., GPU vendors “do not disclose specific architectural details, such as cache parameters or the physical layout of texture data in memory” (ibid p.3). In tandem with the fact that the CPU bandwidth estimate “significantly overestimates” (ibid p.4) true traffic by assuming no register reuse, the methodological asymmetries presented in the paper partially reflect modelling assumptions rather than true architectural ability.

Where the CPU achieves high efficiency through blocking and cache reuse, Fatahalian et al. report that DGEMM on the GPUs they evaluate attains well under 20% of peak FLOPs (Figure 2), despite substantial thread-level parallelism. The bottleneck is that each multiply-accumulate depends on a texture fetch: the work of DGEMM is unchanged (and its span extremely small), but the *memory span* dominates, causing long-latency stalls that parallelism cannot hide [20]. Crucially, these limitations reflect the architectural constraints of GPUs in the early 2000s, rather than fundamental shortcomings of GPU-based DGEMM. Subsequent generations introduced user-controlled shared memory and warp-synchronous execution through CUDA, greatly improving locality support [21]. NVIDIA has also integrated Tensor Cores (specialised fused-multiply-accumulate units designed specifically for GEMM), which are used widely in deep learning and artificial intelligence [22] and are advantageous for the *embarrassingly parallel* tasks (e.g. convolutions, image processing) these areas utilise. As [23] reveals, tensor-core-based DGEMM can deliver up to $5.97\times$ speedup over standard implementations, though these gains rely on the regular, block-structured patterns of dense matrices and degrade significantly under irregularity or sparsity. This addresses a notable point: GEMM performance depends on strong locality and predictable structure, whereas irregular or low-reuse workloads see far less benefit — something Fatahalian et al. fail to touch upon.

As a final remark, contemporary research increasingly scrutinises methods to optimise irregular and sparse matrix–matrix multiplication (SpGEMM) on both CPUs and GPUs, as highlighted in [24], [25] and [26], for instance. Many real-world datasets, such as epidemiological design matrices used in survival analysis and predictive modelling, or document–term matrices used in natural language processing, contain unevenly distributed non-zero entries, with limited opportunities for reuse due to e.g. abundantly missing data. Consequently, memory accesses become scattered and unpredictable, undermining the assumptions that make blocking or tensor-core acceleration effective. Indeed, work on SpGEMM stresses that such patterns exhibit very low spatial and temporal locality and that on-chip caches cannot reduce memory accesses under these irregular distributions (p.478 of [27]). Interestingly, this has prompted research aimed at improving the *algorithms* that such hardware relies on in the first place [28], raising the possibility of rethinking existing architectures altogether. Taken together, these developments make clear that locality remains the decisive factor in modern parallel performance.

References

- [1] Sylvain Arlot and Alain Celisse, *A survey of cross-validation procedures for model selection*, Stat. Surv. **4** (2010), 40–79, DOI 10.1214/09-SS054. MR2602303
- [2] Tom A. Snijders and Theo K. Dijkstra, *On model uncertainty and its statistical implications*, Lecture Notes in Economics and Mathematical Systems **307** (1988).
- [3] SAS Institute Inc., *Forecasting Process Details: ARIMA Models: Notation for ARIMA Models* (2025), https://documentation.sas.com/doc/en/etsug/15.2/etsug_tffordet_sect031.htm. Accessed December 4, 2025.
- [4] Christoph Bergmeir, Mauro Costantini, and José M. Benítez, *On the usefulness of cross-validation for directional forecast evaluation*, Comput. Statist. Data Anal. **76** (2014), 132–143, DOI 10.1016/j.csda.2014.02.001. MR3209431
- [5] László Györfi, Michael Kohler, Adam Krzyżak, and Harro Walk, *A distribution-free theory of non-parametric regression*, Springer Series in Statistics, Springer-Verlag, New York, 2002. MR1920390
- [6] Charles J. Stone, *Consistent Nonparametric Regression*, The Annals of Statistics **5** (1977), no. 4, 595 – 620, DOI 10.1214/aos/1176343886.
- [7] Puning Zhao and Lifeng Lai, *Minimax Regression via Adaptive Nearest Neighbor*, 2019 IEEE International Symposium on Information Theory (ISIT), 2019, pp. 1447–1451, DOI 10.1109/ISIT.2019.8849669.
- [8] Samory Kpotufe, *k-NN Regression Adapts to Local Intrinsic Dimension* (2011), available at <https://arxiv.org/abs/1110.4300>.
- [9] Mona Azadkia, *Optimal choice of k for k-nearest neighbor regression* (2020), available at <https://arxiv.org/abs/1909.05495>.
- [10] Mona Azadkia, *Mona Azadkia: mona.azadkia at stat.math.ethz.ch*, <https://people.math.ethz.ch/~mazadkia/>. Accessed December 5, 2025.
- [11] George Cybenko, *Approximation by superpositions of a sigmoidal function*, Math. Control Signals Systems **2** (1989), no. 4, 303–314, DOI 10.1007/BF02551274. MR1015670
- [12] Kurt Hornik, *Approximation capabilities of multilayer feedforward networks*, Neural Networks **4** (1991), no. 2, 251–257, DOI 10.1016/0893-6080(91)90009-T.
- [13] Kurt Hornik, Maxwell Stinchcombe, and Halbert White, *Multilayer feedforward networks are universal approximators*, Neural Networks **2** (1989), no. 5, 359–366, DOI 10.1016/0893-6080(89)90020-8.
- [14] Tomaso Poggio, Hrushikesh Mhaskar, Lorenzo Rosasco, Brando Miranda, and Qianli Liao, *Why and When Can Deep - but Not Shallow - Networks Avoid the Curse of Dimensionality: a Review* (2017), available at <https://arxiv.org/abs/1611.00740>.
- [15] Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K. Warmuth, *Occam’s razor*, Information Processing Letters **24** (1987), no. 6, 377–380.
- [16] Kayvon Fatahalian, Jeremy Sugerman, and Patrick Hanrahan, *Understanding the efficiency of GPU algorithms for matrix-matrix multiplication*, Proceedings of the ACM SIGGRAPH/EUROGRAPHICS Conference on Graphics Hardware, 2004, pp. 133–137, DOI 10.1145/1058129.1058148.

- [17] Guy E. Blelloch and Bruce M. Maggs, *A Brief Overview of Parallel Algorithms*, <https://www.cs.cmu.edu/~scandal/html-papers/short/short.html>. Accessed December 7, 2025.
- [18] Erik Lindholm, John Nickolls, Stuart Oberman, and John Montrym, *NVIDIA Tesla: A Unified Graphics and Computing Architecture*, *IEEE Micro* **28** (2008), no. 2, 39-55, DOI 10.1109/MM.2008.31.
- [19] Patricia Ortega, *Lecture slides on Parallel Algorithms for Dense Matrix Multiplication: CSE633 Parallel Algorithms*, <https://cse.buffalo.edu/faculty/miller/Courses/CSE633/Ortega-Fall-2012-CSE633.pdf>. Accessed December 8, 2025.
- [20] Dean M. Tullsen and Jeffrey A. Brown, *Handling long-latency loads in a simultaneous multithreading processor*, *Proceedings of the 34th Annual ACM/IEEE International Symposium on Microarchitecture*, 2001, pp. 318-327.
- [21] CORSAIR, *What is a CUDA Core and How Do they Work?*, <https://www.corsair.com/uk/en/explorer/gamer/gaming-pcs/what-is-a-cuda-core-and-how-do-they-work/>. Accessed December 9, 2025.
- [22] NVIDIA, *NVIDIA Tensor Cores: Unprecedented Acceleration for Generative AI*, <https://www.nvidia.com/en-gb/data-center/tensor-cores/>. Accessed December 9, 2025.
- [23] Daichi Mukunoki, Katsuhisa Ozaki, Takeshi Ogita, and Toshiyuki Imamura, *DGEMM using tensor cores, and its accurate and reproducible versions*, *International Conference on High Performance Computing*, 2020, pp. 230-248.
- [24] Cunyang Wei, Haipeng Jia, Yunquan Zhang, Jianyu Yao, Chendi Li, and Wenxuan Cao, *IrGEMM: An Input-Aware Tuning Framework for Irregular GEMM on ARM and X86 CPUs*, *IEEE Transactions on Parallel and Distributed Systems* (2024).
- [25] Thomas McFarland, Julian Bellavita, and Giulia Guidi, *Parallel GPU-Enabled Algorithms for SpGEMM on Arbitrary Semirings with Hybrid Communication*, *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering*, 2025, pp. 216-223.
- [26] Yang Xia, Peng Jiang, Gagan Agrawal, and Rajiv Ramnath, *Scaling sparse matrix multiplication on cpu-gpu nodes*, *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2021, pp. 392-401.
- [27] Ponnuswamy Sadayappan, Bradford Chamberlain, Guido Juckeland, and Hatem Ltaief, *High Performance Computing 35th International Conference, ISC High Performance 2020, Frankfurt/Main, Germany, June 22-25, 2020, Proceedings: 35th International Conference, ISC High Performance 2020, Frankfurt/Main, Germany, June 22-25, 2020, Proceedings*, Springer-Verlag, 2020.
- [28] Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou, *New Bounds for Matrix Multiplication: from Alpha to Omega*, 2023.
- [29] Gebhard Kirchgässner, Jürgen Wolters, and Uwe Hassler, *Introduction to modern time series analysis*, 2nd ed., *Springer Texts in Business and Economics*, Springer, Heidelberg, 2013. MR2986589
- [30] Gérard Biau and Luc Devroye, *Lectures on the nearest neighbor method*, *Springer Series in the Data Sciences*, Springer, Cham, 2015. MR3445317
- [31] David J. Smith and Mavina K. Vamanamurthy, *How small is a unit ball?*, *Math. Mag.* **62** (1989), no. 2, 101-107, DOI 10.2307/2690391. MR0991538

Appendix A

Lemmas

Random k -fold cross-validation bias in forecasting

Lemma A.1. *Let $\{y_t\}_{t=1}^T$ be a univariate time series with $\mathbb{E}[y_t] = \mu_t < \infty$ and $\text{Var}(y_t) = \sigma_t^2 < \infty$. Assume the process is autocorrelated, i.e. $\text{Cov}(y_t, y_{t+\tau}) \neq 0$, for some $\tau > 0$. If the test set contains future observations, then random k -fold cross-validation (CV) underestimates the true forecasting error:*

$$\text{MSE}_{\text{CV}}(t) < \text{MSE}_{\text{fore}}(t).$$

Proof. For genuine forecasting, the (Bayes) optimal predictor (see p.78 of [29]) uses only *past* information:

$$\hat{y}_t^{\text{fore}} = \mathbb{E}[y_t \mid y_1, \dots, y_{t-1}].$$

Its mean squared error is defined as

$$\text{MSE}_{\text{fore}}(t) := \mathbb{E}[(y_t - \hat{y}_t^{\text{fore}})^2]. \quad (1)$$

Randomly partition the indices into folds $\mathcal{F}_1, \dots, \mathcal{F}_k$. Suppose $y_t \in \mathcal{F}_j$, so that the CV training set is

$$\mathcal{D}_{\text{train}}^{(t)} := \{y_s : s \notin \mathcal{F}_j\}.$$

This set can contains both past and future values relative to t . The CV predictor is

$$\hat{y}_t^{\text{CV}} = \mathbb{E}[y_t \mid \mathcal{D}_{\text{train}}^{(t)}],$$

with

$$\text{MSE}_{\text{CV}}(t) := \mathbb{E}[(y_t - \hat{y}_t^{\text{CV}})^2]. \quad (2)$$

By the triangle equality (conditioning on $\mathcal{D}_{\text{train}}^{(t)}$),

$$\mathbb{E}[(y_t - \hat{y}_t^{\text{fore}})^2 \mid \mathcal{D}_{\text{train}}^{(t)}] \leq \mathbb{E}[(y_t - \hat{y}_t^{\text{CV}})^2 \mid \mathcal{D}_{\text{train}}^{(t)}] + \mathbb{E}[(\hat{y}_t^{\text{CV}} - \hat{y}_t^{\text{fore}})^2 \mid \mathcal{D}_{\text{train}}^{(t)}].$$

Thus, to prove that the inequality is strict, it suffices to show the last term is strictly positive.

Define the event that at least one future observation enters the CV training set:

$$\mathcal{A} := \{\exists s > t : s \notin \mathcal{F}_j\}.$$

Its complement is

$$\mathcal{A}^c := \{s > t \implies s \in \mathcal{F}_j\},$$

meaning all future points lie in the same fold.

Since each of the $T - t$ future points independently enters $\mathcal{F}_j$ with probability $1/k$,

$$\mathbb{P}(\mathcal{A}^c) = (1/k)^{T-t}, \quad \mathbb{P}(\mathcal{A}) = 1 - (1/k)^{T-t} > 0.$$

With positive probability, (random) CV includes some future values in the training set; as $T \rightarrow \infty$, this occurs almost surely.

Because the series is autocorrelated, conditioning on any future value y_s with $s > t$ alters the optimal conditional expectation:

$$\hat{y}_t^{\text{CV}} \neq \hat{y}_t^{\text{fore}} \quad \text{with positive probability.}$$

Therefore,

$$(\hat{y}_t^{\text{CV}} - \hat{y}_t^{\text{fore}})^2 > 0 \quad \text{on event } \mathcal{A},$$

so

$$\mathbb{E} [(\hat{y}_t^{\text{CV}} - \hat{y}_t^{\text{fore}})^2] > 0. \tag{5}$$

Substituting (5) into (4) gives

$$\begin{aligned} \text{MSE}_{\text{CV}}(t) &= \text{MSE}_{\text{fore}}(t) - \mathbb{E} [(\hat{y}_t^{\text{CV}} - \hat{y}_t^{\text{fore}})^2] \\ &< \text{MSE}_{\text{fore}}(t). \end{aligned}$$

Thus, random k -fold cv yields optimistically biased forecast error estimates:

$$\boxed{\text{MSE}_{\text{CV}}(t) < \text{MSE}_{\text{fore}}(t)}.$$

□

Asymptotic behaviour of expected mean squared error

Lemma A.2. *Let $\mathbf{X}$ be a random vector, $\mathbf{X} \in [0, 1]^d$ for simplicity, where $\mathbf{X}_{(k)}(\mathbf{x})$ denote the k^{th} nearest neighbour of $\mathbf{x}$. Assume also that we choose k such that, when $k \rightarrow \infty$, $k/n \rightarrow 0$ when $n \rightarrow \infty$ (universal consistency). Then, as $d \rightarrow \infty$,*

$$\mathbb{E} [\|\mathbf{X}_{(k)}(\mathbf{x}) - \mathbf{X}\|^2] = O(d^2).$$

Proof. By Theorem 2.4 of [30],

$$\mathbb{E} [\|\mathbf{X}_{(k)}(\mathbf{x}) - \mathbf{X}\|^2] \leq c'_d \left(\frac{k}{n}\right)^{2/d},$$

where

$$V_d = \frac{\pi^{d/2}}{d/2 \cdot \Gamma(d/2)} \quad (1)$$

denotes the volume of the unit (Euclidean) ball in $\mathbb{R}^d$ and

$$c'_d = \frac{2^{3+2/d} (1 + \sqrt{d})^2}{V_d^{2/d}}.$$

By the assumptions stated in the lemma, we see that, as d becomes infinitely large,

$$\left(\frac{k}{n}\right)^{2/d} \rightarrow 1.$$

Using Stirling's approximation,

$$\Gamma(d/2) \sim \sqrt{2\pi} \cdot (d/2)^{(d-1)/2} e^{-d/2}. \quad (2)$$

Inserting (2) into (1) yields

$$V_d \sim \frac{(\pi e)^{d/2}}{\sqrt{2\pi} \cdot (d/2)^{(d+1)/2}}. \quad (3)$$

Raising (3) to the power of $2/d$, we see that

$$V_d^{2/d} \sim \frac{\pi e}{(2\pi)^{1/d} (d/2)^{1+1/d}}. \quad (4)$$

Consider the term $(d/2)^{1+1/d}$:

$$\begin{aligned} (d/2)^{1+1/d} &= (d/2)(d/2)^{1/d} \\ &= (d/2) \cdot \exp(\log(d/2)/d) && \text{(exponent rule)} \\ &= (d/2) \cdot (1 + \log(d/2)/d + o(\log(d/2)/d)) && \text{(Taylor expansion)} \\ &\sim (d/2) \cdot (1 + o(1)) && (\log(d/2)/d \rightarrow 0) \\ &\sim d/2 && (5) \end{aligned}$$

It is easy to deduce $(2\pi)^{1/d} \rightarrow 1$. Plugging this term and the term from (5) into (4),

$$V_d^{2/d} \sim \frac{2\pi e}{d} = O(1/d).$$

As $d \rightarrow \infty$, $V_d^{2/d} \rightarrow 0$, an established result [31].

Furthermore, $2^{3+2/d} = O(1)$ and $(1 + \sqrt{d})^2 = 1 + 2\sqrt{d} + d = O(d)$, so

$$\boxed{c'_d = O(d/(1/d)) \cdot O(1) = O(d^2)}.$$

□

Appendix B

Figures

Time series diagnostic plots

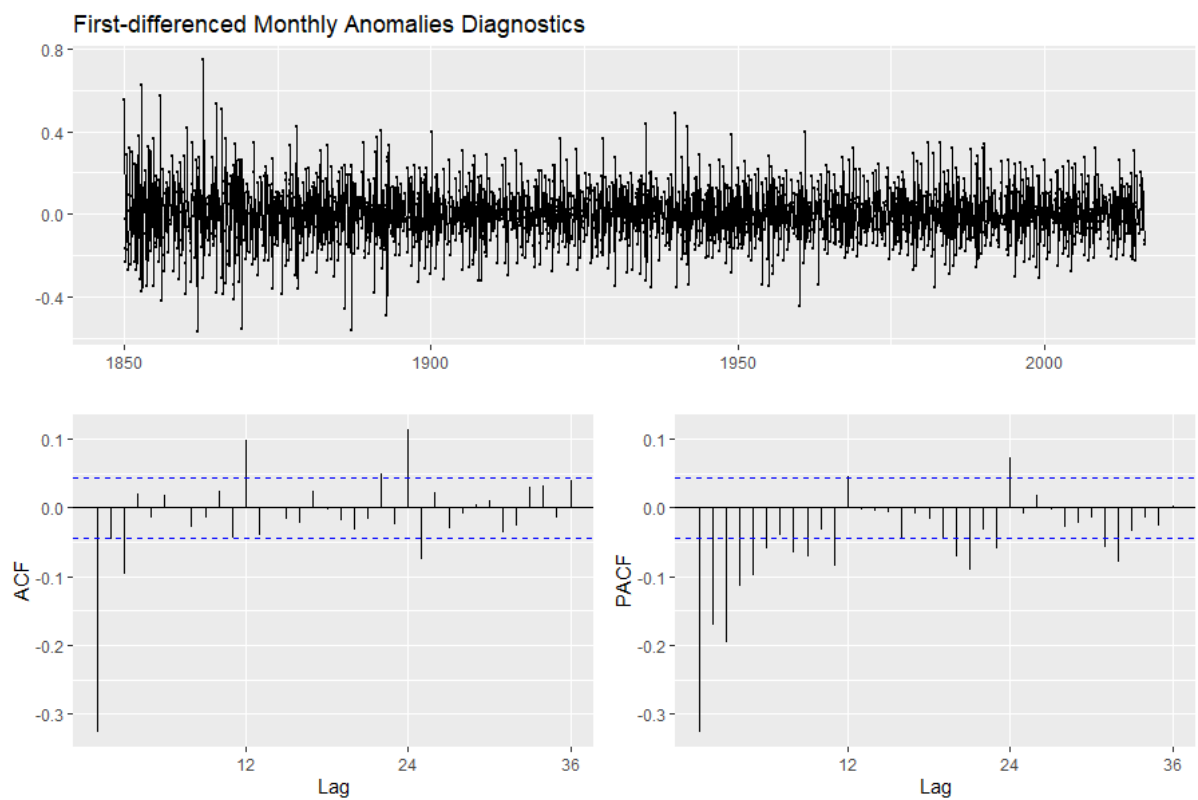


Figure B.1: First-differenced monthly temperature anomalies with corresponding (partial) autocorrelation function plots. The differenced series exhibits a large, negative spike at lag 1, consistent with an MA(1) term, and significant seasonal autocorrelations at lags 12 and 24, indicating a seasonal MA(2) component.

Blocked cross-validation model comparison

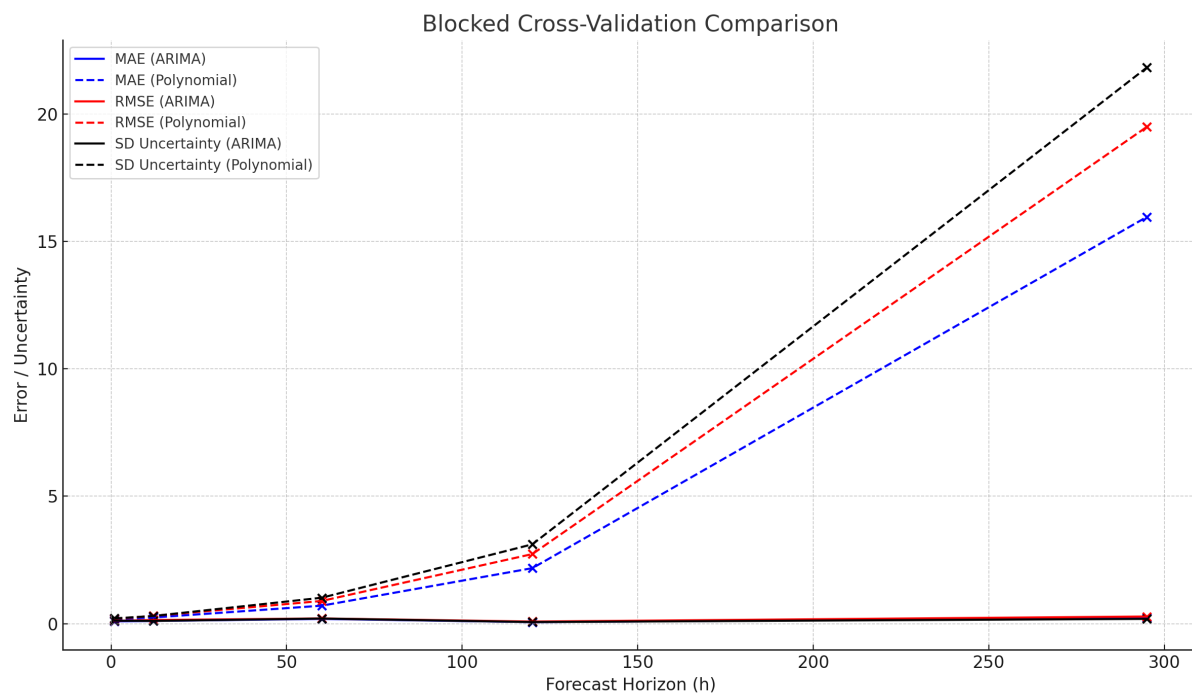


Figure B.2: An interpolated scatter plot of blocked cross-validation performance across increasing forecast horizons. Errors and empirical uncertainty are reported for two competing models: an $\text{ARIMA}(2,1,1)(0,0,2)_{12}$ model and a fifth-degree polynomial regression model.

Model forecast comparison

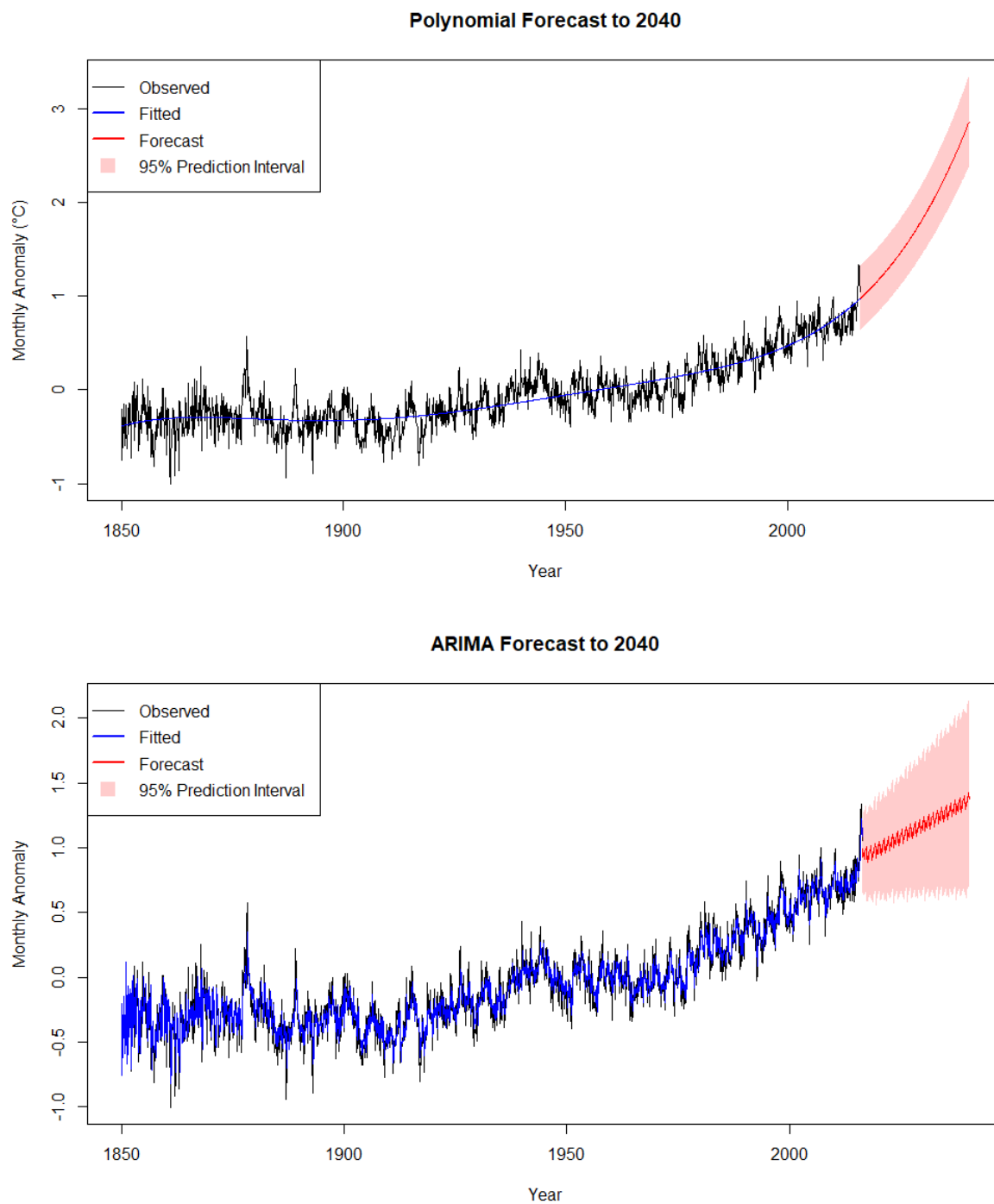


Figure B.3: Run-sequence plots comparing long-term forecasts of monthly temperature anomalies to (the end of) 2040, with 95% prediction intervals shaded in pink, for two competing models: an $\text{ARIMA}(2,1,1)(0,0,2)_{12}$ model and a fifth-degree polynomial regression model.

k -nearest neighbour regression with optimal k

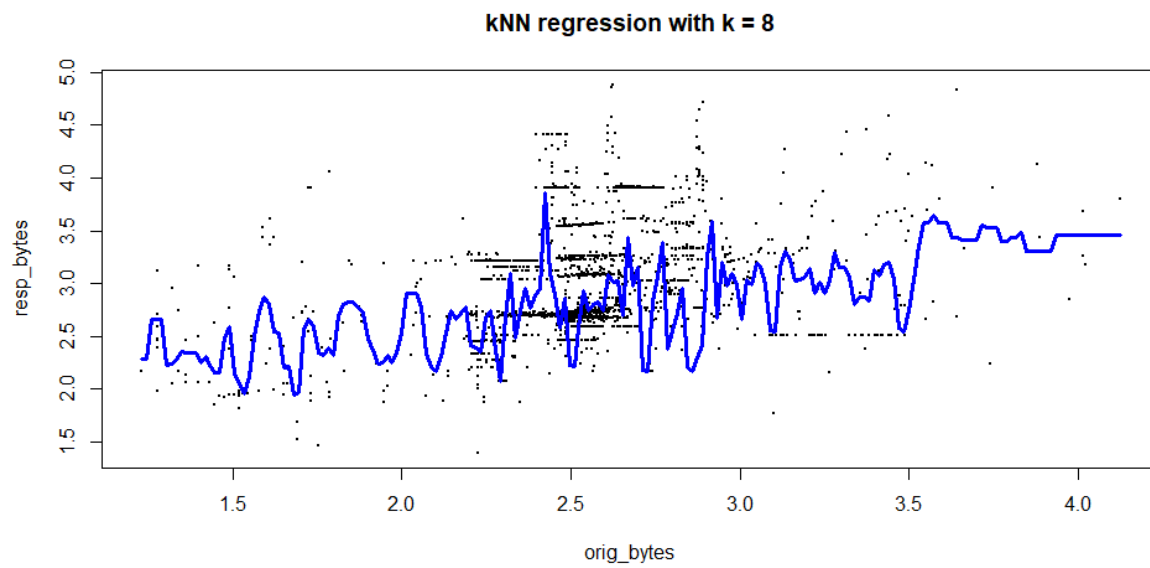


Figure B.4: Estimated regression function $m(\mathbf{x})$ obtained via k -nearest neighbour regression with $k^* = 8$, selected by leave-one-out cross-validation. The fitted curve (blue) is plotted against the observed $(\text{orig_bytes}, \text{resp_bytes})$ pairs.

Universal Approximation Theorem results

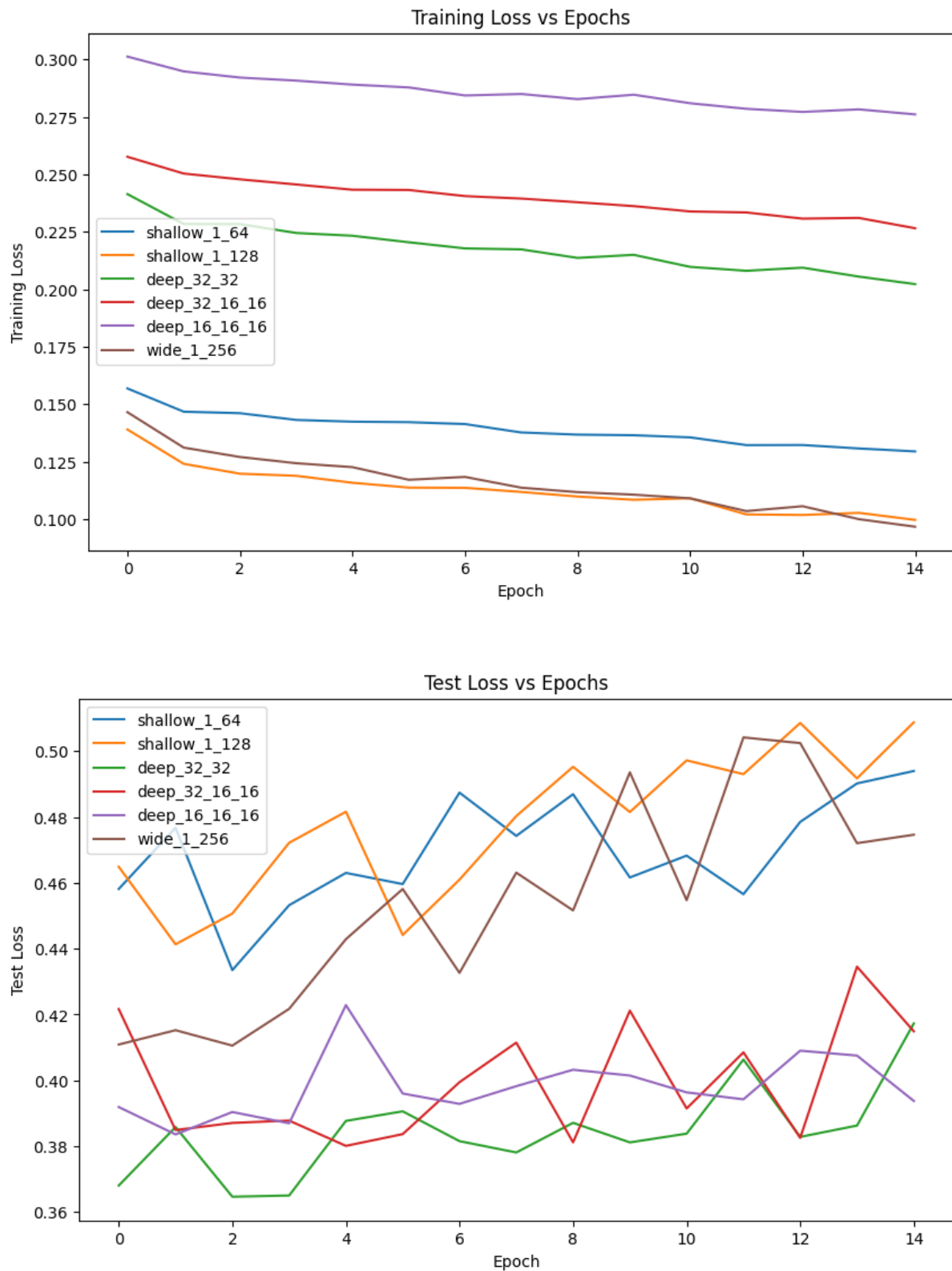


Figure B.5: Training and test loss over 15 epochs for six architectures. Wider, shallow networks converge faster and to lower training loss values, whereas deeper models show slower convergence and higher final training loss. Wider, shallow networks, contrastingly, exhibit consistently higher test loss, perhaps with slightly slower convergence.

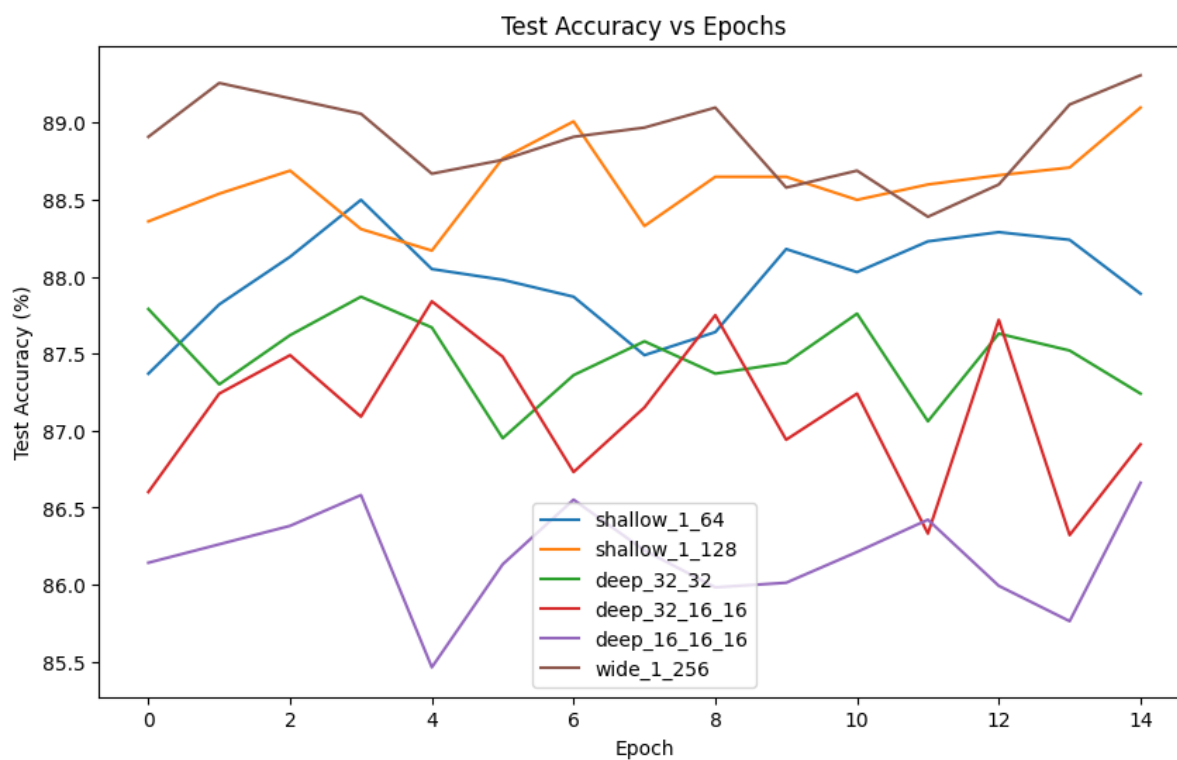


Figure B.6: Test accuracy over 15 epochs for six architectures. Wider, shallow networks achieve the highest test accuracies overall.

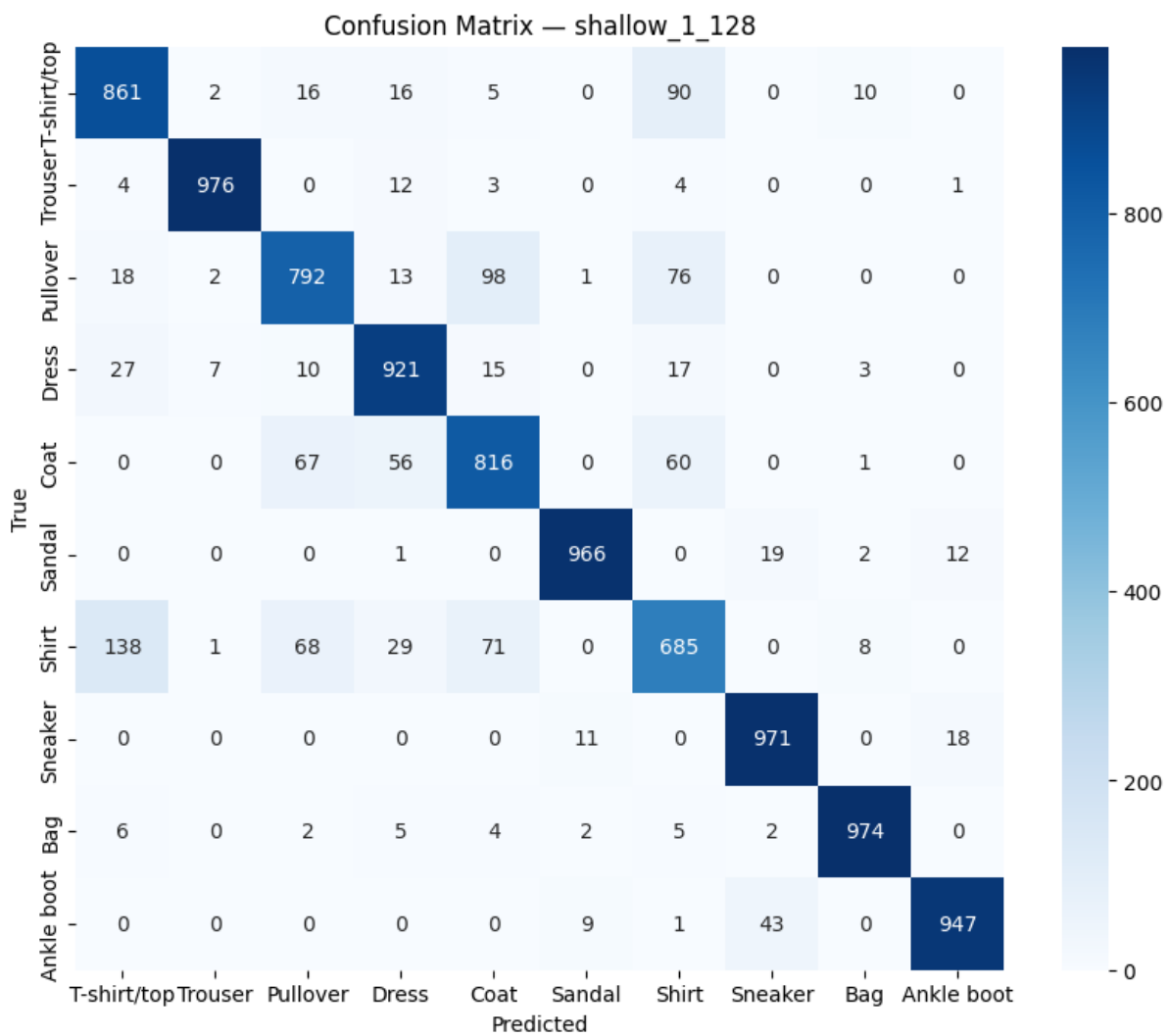


Figure B.7: Confusion matrix for the selected 1×128 model. Errors occur primarily between visually similar classes, while distinct categories, such as trousers and ankle boots, are classified with high confidence.

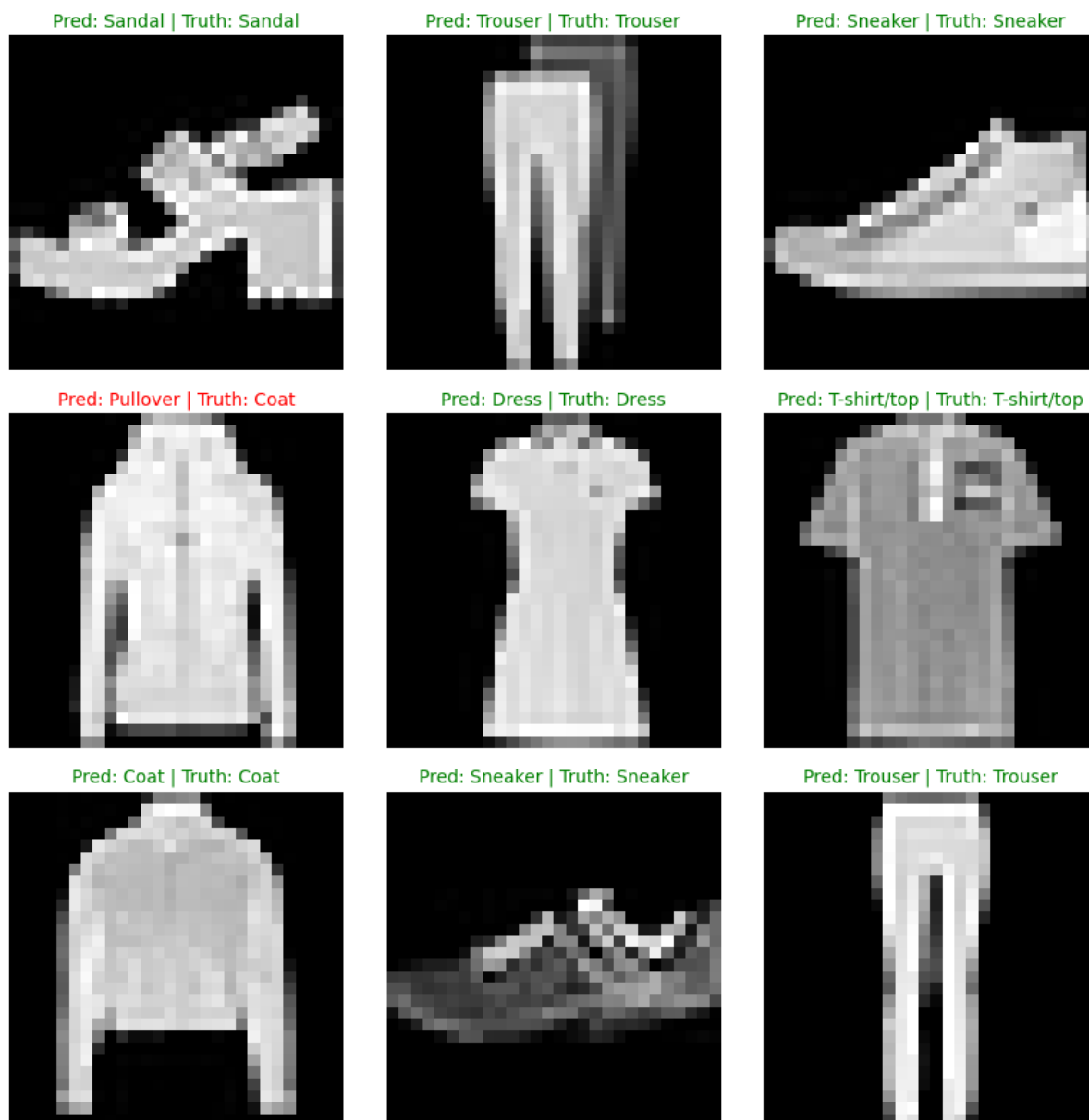


Figure B.8: Example predictions from the final 1×128 model. Correct classifications appear in green; misclassifications (e.g. pullover misidentified as coat) arise mostly in categories with overlapping visual features.

Appendix C

R code

Blocked cross-validation

```
# ===== #
# ARIMA model: Blocked cross-validation #
# ===== #

library(forecast)

# my_ts: given time series
# n_blocks: number of blocks
# h: forecast horizon

blocked_cv <- function(my_ts, n_blocks, h) {
  n <- length(my_ts)
  block_size <- floor(n / n_blocks)

  errors <- c() # empty vector to store errors

  # Looping over each block
  for (b in 1:n_blocks) {
    start <- (b - 1) * block_size + 1
    end <- b * block_size
    block <- my_ts[start:end]

    # Skipping if there is too little future data
    if ((end + h) > n) next

    # Fitting proposed ARIMA model on current training block
    fit <- Arima(block, order = c(2,1,1),
                  seasonal = list(order = c(0,0,2), period = 12))

    # Forecasting h steps ahead
    fc <- forecast(fit, h = h)$mean

    # h-step-ahead errors
    actual_h <- my_ts[end + h]
    errors <- c(errors, actual_h - fc[h])
  }

  # Performance metrics for chosen h
  MAE <- mean(abs(errors))
}
```

```

RMSE <- sqrt(mean(errors^2))
SD_Uncertainty <- sd(errors)

return(c(MAE = MAE, RMSE = RMSE, SD_Uncertainty = SD_Uncertainty))
}

# === Comparing varying h === #

library(doParallel)
library(foreach)

cores <- detectCores() - 1 # spare CPU core for background processes
cl <- makeCluster(cores)
registerDoParallel(cl)

hs <- c(1, 12, 60, 120, 295) # candidate forecast horizons

# Performing parallel blocked CV for each h and stacking results
blocked_results_arima <- foreach(h = hs, .combine = rbind,
                                .packages = c("forecast")) %dopar% {
  blocked_cv(my_ts, n_blocks = 5, h = h)
}

stopCluster(cl)

rownames(blocked_results_arima) <- paste0("h=", hs)
colnames(blocked_results_arima) <- c("MAE", "RMSE", "SD_Uncertainty")

```

k -nearest neighbour regression with optimal k

```

#####
# LOOCV using knnopt #
#####

# Credits to Mona Azadkia for the knnopt function:
# https://people.math.ethz.ch/~mazadkia/

library(RANN)

# Loading the data
source("https://raw.githubusercontent.com/dsbristol/dst/master/code/loadconndata.R")

testdata=conndata[,c("orig_bytes","resp_bytes",
                    "orig_ip_bytes","resp_ip_bytes")]
testdata[testdata=="-"]=0
testdata[testdata=="0"]=0
for(i in 1:4) testdata[,i]=log10(as.numeric(testdata[,i])+1)
rownames(testdata)=NULL

# Restricting the data to successful flows
nz <- which(testdata$orig_bytes > 0 & testdata$resp_bytes > 0)

x <- as.matrix(testdata[nz, "orig_bytes"])
y <- testdata[nz, "resp_bytes"]

xy <- cbind(x, y)
md <- mahalanobis(xy, colMeans(xy), cov(xy)) # Mahalanobis distance

```

```
sig_lvl <- qchisq(0.975, df = 2) # 95% confidence level ellipse
outliers <- which(md > sig_lvl) # outliers lie outside the ellipse

keep <- setdiff(1:nrow(xy), outliers) # keeping non-outliers

x <- as.matrix(x[keep])
y <- y[keep]

# Creating a grid for plotting
xnew <- as.matrix(seq(min(x), max(x), length.out = 200), ncol = 1)

set.seed(1) # setting seed due to random tie-breaking

pred <- predict.knnopt(x, y, xnew, binary = FALSE, k_max = 100)
k_opt <- pred$kopt
k_preds <- as.numeric(pred$predict)
k_mse <- pred$MSE

# === Bootstrap analysis of optimal k === #

library(doParallel)
library(foreach)

B <- 100 # 100 resamples
cl <- makeCluster(detectCores() - 1)
registerDoParallel(cl)

k_boot <- foreach(b = 1:B, .combine = c, .packages = "RANN") %dopar% {

  idx <- sample(seq_along(y), replace = TRUE)

  x_boot <- x[idx, , drop = FALSE]
  y_boot <- y[idx]

  knnopt(x_boot, y_boot, binary = FALSE, k_max = 20)$kopt
}

stopCluster(cl)
```